

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П.КОРОЛЕВА»

Институт «Информатики и кибернетики»

Специальность «Фотоника и оптоинформатика»

Отчет по лабораторной работе № 7

Выполнила: Гамаюнова Д.И.,

группа 6201-120303D

Проверил: преподаватель Борисов Д.С.

Самара 2025

Задание 1.

В интерфейсе TabulatedFunction добавила необходимый родительский тип.

```
package functions;

import java.util.Iterator;

public interface TabulatedFunction extends Function, Cloneable, Iterable<FunctionPoint> { 18 usages
    Object clone(); 2 implementations
    int getPointsCount(); 9 usages 2 implementations
    FunctionPoint getPoint(int index); 2 usages 2 implementations
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException; no usages
    double getPointX(int index); 5 usages 2 implementations
    void setPointX(int index, double x) throws InappropriateFunctionPointException; no usages 2 implementations
    double getPointY(int index); 2 usages 2 implementations
    void setPointY(int index, double y); 1 usage 2 implementations
    void deletePoint(int index); no usages 2 implementations
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException; no usages 2 implementations
}
```

В классах, реализующих интерфейс TabulatedFunction, добавила требующийся метод, возвращающий объект итератора. Метод удаления всегда выбрасывает исключение UnsupportedOperationException. Метод получения следующего элемента выбрасывает исключение NoSuchElementException, если следующего элемента нет.

ArrayTabulatedFunction:

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int currentIndex = 0; 2 usages

        @Override
        public boolean hasNext() {
            return currentIndex < size;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            // Возвращаем копию точки, чтобы не нарушать инкапсуляцию
            return new FunctionPoint(points[currentIndex++]);
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}
```

LinkedListTabulatedFunction:

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode currentNode = head.next; 3 usages
        private int currentIndex = 0; 2 usages

        @Override
        public boolean hasNext() {
            return currentIndex < size;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            // Возвращаем копию точки, чтобы не нарушать инкапсуляцию
            FunctionPoint point = new FunctionPoint(currentNode.point);
            currentNode = currentNode.next;
            currentIndex++;
            return point;
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}
```

В методе main() программы проверила работу итераторов классов табулированных функций.

```
import functions.*;
import functions.basic.*;
import java.io.*;

public class Main {
    public static void main(String[] args) {
        System.out.println("Задание 1.");
        System.out.println("Тестирование итератора ArrayTabulatedFunction:");
        FunctionPoint[] points1 = {
            new FunctionPoint(x: 0, y: 0),
            new FunctionPoint(x: 1, y: 1),
            new FunctionPoint(x: 2, y: 4)
        };
        TabulatedFunction f1 = new ArrayTabulatedFunction(points1);

        for (FunctionPoint p : f1) {
            System.out.println(p);
        }

        System.out.println("\nТестирование итератора LinkedListTabulatedFunction:");
        TabulatedFunction f2 = new LinkedListTabulatedFunction(points1);

        for (FunctionPoint p : f2) {
            System.out.println(p);
        }
    }
}
```

Задание2.

В пакете functions описала базовый интерфейс фабрик табулированных функций TabulatedFunctionFactory.

```
package functions;

public interface TabulatedFunctionFactory { no usages 2 implementations
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points); no usages 2 implementations
}
```

Для удобства описала в классах ArrayTabulatedFunction и LinkedListTabulatedFunction классы фабрик ArrayTabulatedFunctionFactory и LinkedListTabulatedFunctionFactory, реализующие интерфейс фабрики и порождающие объекты соответствующих классов табулированных функций. Сделала классы фабрик вложенными и публичными.

ArrayTabulatedFunction:

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { 1 usage
    @Override 2 usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }
    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }
    @Override 3 usages
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

LinkedListTabulatedFunction:

```
public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory { no usages
    @Override 2 usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }
    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }
    @Override 3 usages
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}
```

В классе `TabulatedFunctions` объявила приватное статическое поле типа `TabulatedFunctionFactory` и проинициализировала его объектом одного из описанных классов фабрик. Также объявила метод `setTabulatedFunctionFactory()`, позволяющий заменить объект фабрики. Ещё в классе `TabulatedFunctions` описала три перегруженных метода `TabulatedFunction` `createTabulatedFunction()`, возвращающих объекты табулированных функций, созданные с помощью текущей фабрики.

```
public final class TabulatedFunctions { 1 usage
    private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();

    private TabulatedFunctions() { no usages
        throw new AssertionError("Нельзя создать экземпляр класса TabulatedFunctions");
    }

    public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) { no usages
        TabulatedFunctions.factory = factory;
    }

    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) { no usage
        return factory.createTabulatedFunction(leftX, rightX, pointsCount);
    }

    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) { no usage
        return factory.createTabulatedFunction(leftX, rightX, values);
    }

    public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { no usages
        return factory.createTabulatedFunction(points);
    }
}
```

В остальных методах класса, где требуется создание объектов табулированных функций, заменила явное создание объектов с помощью конструкторов на вызов соответствующего метода `createTabulatedFunction()`.

```
public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) { 2 usages
    if (function == null) {
        throw new IllegalArgumentException("Функция не может быть null");
    }

    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не менее 2: " + pointsCount);
    }

    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница должна быть меньше правой: " + leftX + " >= " + rightX);
    }

    if (leftX < function.getLeftDomainBorder()) {
        throw new IllegalArgumentException("Левая граница табулирования выходит за область определения функции: " +
            leftX + " < " + function.getLeftDomainBorder());
    }

    if (rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Правая граница табулирования выходит за область определения функции: " +
            rightX + " > " + function.getRightDomainBorder());
    }

    TabulatedFunction tabulatedFunc = factory.createTabulatedFunction(leftX, rightX, pointsCount);
    for (int i = 0; i < pointsCount; i++) {
        double x = tabulatedFunc.getPointX(i);
        double y = function.getFunctionValue(x);
        tabulatedFunc.setPointY(i, y);
    }

    return tabulatedFunc;
}
```

```

public static TabulatedFunction inputTabulatedFunction(InputStream in) throws IOException { no usages
    if (in == null) {
        throw new IllegalArgumentException("Входной поток не может быть null");
    }
    DataInputStream dataIn = new DataInputStream(in);
    int pointsCount = dataIn.readInt();
    if (pointsCount < 2) {
        throw new IOException("Некорректное количество точек: " + pointsCount);
    }
    FunctionPoint[] points = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        double x = dataIn.readDouble();
        double y = dataIn.readDouble();
        points[i] = new FunctionPoint(x, y);
    }
    return factory.createTabulatedFunction(points);
}

```

```

public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException { no usages
    if (in == null) {
        throw new IllegalArgumentException("Входной поток не может быть null");
    }
    StreamTokenizer tokenizer = new StreamTokenizer(in);
    if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) {
        throw new IOException("Ожидалось количество точек");
    }
    int pointsCount = (int) tokenizer.nval;
    if (pointsCount < 2) {
        throw new IOException("Некорректное количество точек: " + pointsCount);
    }
    FunctionPoint[] points = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) {
            throw new IOException("Ожидалась координата x для точки " + i);
        }
        double x = tokenizer.nval;
        if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) {
            throw new IOException("Ожидалась координата y для точки " + i);
        }
        double y = tokenizer.nval;
        points[i] = new FunctionPoint(x, y);
    }
    return factory.createTabulatedFunction(points);
}

```

В методе main() проверила работу фабрик:

```

System.out.println("\nЗадание 2.");
Function f = new functions.basic.Cos();
TabulatedFunction tf;
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Фабрика по умолчанию: " + tf.getClass());
TabulatedFunctions.setTabulatedFunctionFactory(
    new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Фабрика LinkedListTabulatedFunction: " + tf.getClass());
TabulatedFunctions.setTabulatedFunctionFactory(
    new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Фабрика ArrayTabulatedFunction: " + tf.getClass());

```

Задание 3.

В классе TabulatedFunctions добавила ещё три перегруженных версии метода createTabulatedFunction().

```
public static TabulatedFunction createTabulatedFunction( 2 usages
    Class<? extends TabulatedFunction> functionClass,
    double leftX, double rightX, int pointsCount) {
    try {
        Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(double.class, double.class, int.class);
        return constructor.newInstance(leftX, rightX, pointsCount);
    } catch (NoSuchMethodException | SecurityException |
        InstantiationException | IllegalAccessException |
        IllegalArgumentException | InvocationTargetException e) {
        throw new IllegalArgumentException("Cannot create tabulated function", e);
    }
}

public static TabulatedFunction createTabulatedFunction( 1 usage
    Class<? extends TabulatedFunction> functionClass,
    double leftX, double rightX, double[] values) {
    try {
        Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(double.class, double.class, double[].class);
        return constructor.newInstance(leftX, rightX, values);
    } catch (NoSuchMethodException | SecurityException |
        InstantiationException | IllegalAccessException |
        IllegalArgumentException | InvocationTargetException e) {
        throw new IllegalArgumentException("Cannot create tabulated function", e);
    }
}

public static TabulatedFunction createTabulatedFunction( 1 usage
    Class<? extends TabulatedFunction> functionClass,
    FunctionPoint[] points) {
    try {
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(FunctionPoint[].class);
        return constructor.newInstance((Object) points);
    } catch (NoSuchMethodException | SecurityException |
        InstantiationException | IllegalAccessException |
        IllegalArgumentException | InvocationTargetException e) {
        throw new IllegalArgumentException("Cannot create tabulated function", e);
    }
}
```

В классе TabulatedFunctions перегрузила методы, создающие объекты табулированных функций, добавив версии, принимающие также ссылку типа Class на описание класса, объект которого требуется создать.

```
public static TabulatedFunction tabulate( 3 usages
    Class<? extends TabulatedFunction> functionClass, Function function, double leftX, double rightX, int pointsCount) {
    if (function == null) {
        throw new IllegalArgumentException("Функция не может быть null");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не менее 2: " + pointsCount);
    }
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница должна быть меньше правой: " + leftX + " >= " + rightX);
    }
    if (leftX < function.getLeftDomainBorder()) {
        throw new IllegalArgumentException("Левая граница табулирования выходит за область определения функции: " +
            leftX + " < " + function.getLeftDomainBorder());
    }
    if (rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Правая граница табулирования выходит за область определения функции: " +
            rightX + " > " + function.getRightDomainBorder());
    }
    TabulatedFunction tabulatedFunc = createTabulatedFunction(functionClass, leftX, rightX, pointsCount);
    for (int i = 0; i < pointsCount; i++) {
        double x = tabulatedFunc.getPointX(i);
        double y = function.getFunctionValue(x);
        tabulatedFunc.setPointY(i, y);
    }
    return tabulatedFunc;
}
```



```

public static TabulatedFunction tabulate( no usages
    Class<? extends TabulatedFunction> functionClass, Function function, double leftX, double rightX) {
    int pointsCount = (int) Math.max(2, Math.ceil((rightX - leftX) * 10) + 1);
    return tabulate(functionClass, function, leftX, rightX, pointsCount);
}

public static TabulatedFunction tabulate( no usages
    Class<? extends TabulatedFunction> functionClass, Function function, int pointsCount) {
    if (function == null) {
        throw new IllegalArgumentException("Функция не может быть null");
    }
    double leftX = function.getLeftDomainBorder();
    double rightX = function.getRightDomainBorder();
    if (Double.isInfinite(leftX) || Double.isInfinite(rightX)) {
        throw new IllegalArgumentException("Нельзя табулировать функцию с бесконечной областью определения: [" +
            leftX + ", " + rightX + "]");
    }
    return tabulate(functionClass, function, leftX, rightX, pointsCount);
}
}

```

```

public static TabulatedFunction inputTabulatedFunction(InputStream in, no usages
    Class<? extends TabulatedFunction> functionClass) throws IOException {
    DataInputStream dataIn = new DataInputStream(in);
    int pointsCount = dataIn.readInt();
    FunctionPoint[] points = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        points[i] = new FunctionPoint(dataIn.readDouble(), dataIn.readDouble());
    }
    return createTabulatedFunction(functionClass, points);
}

public static TabulatedFunction readTabulatedFunction(Reader in, no usages
    Class<? extends TabulatedFunction> functionClass) throws IOException {
    StreamTokenizer tokenizer = new StreamTokenizer(in);
    tokenizer.parseNumbers();
    if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) throw new IOException("Ошибка формата");
    int pointCount = (int) tokenizer.nval;
    FunctionPoint[] points = new FunctionPoint[pointCount];
    for (int i = 0; i < pointCount; i++) {
        if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) throw new IOException("Ошибка X");
        double x = tokenizer.nval;
        if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) throw new IOException("Ошибка Y");
        double y = tokenizer.nval;
        points[i] = new FunctionPoint(x, y);
    }
    return createTabulatedFunction(functionClass, points);
}
}

```

Проверила в методе main() работу методов рефлексивного создания объектов, а также методов класса TabulatedFunctions, использующих создание объектов.

```

System.out.println("\n\nЗадание 3.");
TabulatedFunction f3 = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3);
System.out.println("Тип: " + f3.getClass());
System.out.println("Функция: " + f3);

TabulatedFunction f4 = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[] {0, 5, 10});
System.out.println("\n\nТип: " + f4.getClass());
System.out.println("Функция: " + f4);

FunctionPoint[] points = {
    new FunctionPoint(x: 0, y: 0),
    new FunctionPoint(x: 10, y: 10)
};
TabulatedFunction f5 = TabulatedFunctions.createTabulatedFunction(
    LinkedListTabulatedFunction.class, points);
System.out.println("\n\nТип: " + f5.getClass());
System.out.println("Функция: " + f5);

TabulatedFunction f6 = TabulatedFunctions.tabulate(
    LinkedListTabulatedFunction.class, new functions.basic.Sin(), leftX: 0, Math.PI, pointsCount: 11);
System.out.println("\n\nТип: " + f6.getClass());
System.out.println("Функция: " + f6);

```


Результаты

Задание 1.

Тестирование итератора ArrayTabulatedFunction:

(0,00; 0,00)

(1,00; 1,00)

(2,00; 4,00)

Тестирование итератора LinkedListTabulatedFunction:

(0,00; 0,00)

(1,00; 1,00)

(2,00; 4,00)

Задание 2.

Фабрика по умолчанию: class functions.ArrayTabulatedFunction

Фабрика LinkedListTabulatedFunction: class functions.LinkedListTabulatedFunction

Фабрика ArrayTabulatedFunction: class functions.ArrayTabulatedFunction

Задание 3.

Тип: class functions.ArrayTabulatedFunction

Функция: {(0,00; 0,00), (5,00; 0,00), (10,00; 0,00)}

Тип: class functions.ArrayTabulatedFunction

Функция: {(0,00; 0,00), (5,00; 5,00), (10,00; 10,00)}

Тип: class functions.LinkedListTabulatedFunction

Функция: {(0,00; 0,00), (10,00; 10,00)}

Тип: class functions.LinkedListTabulatedFunction

Функция: {(0,00; 0,00), (0,31; 0,31), (0,63; 0,59), (0,94; 0,81), (1,26; 0,95), (1,57; 1,00), (1,88; 0,95), (2,20; 0,81), (2,51; 0,59), (2,83; 0,31), (3,14; 0,00)}